

7. Not Only the Simplex Method

Tens of different algorithms have been suggested for linear programming over the years. Most of them didn't work very well, and only very few have turned out as serious competitors to the simplex method, the historically first algorithm. But at least two methods raised great excitement at the time of discovery and they are surely worth mentioning.

The first of them, the *ellipsoid method*, cannot compete with the simplex method in practice, but it had immense theoretical significance. It is the first linear programming algorithm for which it was proved that it always runs in polynomial time (which is not known about the simplex method up to the present, and for many pivot rules it is not even true).

The second is the *interior point method*, or rather, we should say interior point *methods*, since it is an entire group of algorithms. For some of them a polynomial bound on the running time has also been proved, but moreover, these algorithms successfully compete with the simplex method in practice. It seems that for some types of linear programs the simplex method is better, while for others interior point methods are the winners.

Let us remark that several other algorithms, closely related to the simplex method, are used for linear programming as well. The *dual simplex method* can roughly be described as the simplex method applied to the dual linear program. But details of the implementation, which are crucial for the speed of the algorithm in practice, are somewhat different. The dual simplex method is particularly suitable for linear programs that in equational form have $n - m$ significantly smaller than m .

The *primal-dual method* goes through a sequence of feasible solutions of the dual linear program. To get from one such solution to the next, it does not perform a pivot step, but it solves an auxiliary problem that may be derived from the primal linear program or by other means. This greater freedom can be useful, for instance, in approximation algorithms for combinatorial optimization problems.

A little more about the dual simplex method and the primal-dual method can be found in the glossary.

7.1 The Ellipsoid Method

The ellipsoid method was invented in 1970 by Shor, Judin, and Nemirovski as an algorithm for certain nonlinear optimization problems. In 1979 Leonid Khachyan outlined, in a short note, how linear programs can be solved by this method in provably polynomial time. The world press made a sensation out of this since the journalists contorted the result and presented it as an unprecedented breakthrough in practical computational methods (giving the Soviets a technological edge over the West...).¹ However, the ellipsoid method has never been interesting for the practice of linear programming—Khachyan’s discovery was indeed extremely significant, but for the theory of computational complexity. It solved an open problem that many people had attacked in vain for many years. The solution was conceptually utterly different from previous approaches, which were mostly variations of the simplex method.

Input size and polynomial algorithms. In order to describe what we mean by a polynomial algorithm for linear programming, we have to define the **input size** of a linear program. Roughly speaking, it is the total number of bits needed for writing down the input to a linear programming algorithm.

First we define the *bit size* of an integer i as

$$\langle i \rangle = \lceil \log_2(|i| + 1) \rceil + 1,$$

which is the number of bits of i written in binary, including one bit for the sign. For a rational number r , i.e., a fraction $r = p/q$, the bit size is defined as $\langle r \rangle = \langle p \rangle + \langle q \rangle$. For an n -component rational vector $\mathbf{v}$ we put $\langle \mathbf{v} \rangle = \sum_{i=1}^n \langle v_i \rangle$,

¹ The following quotation from

E. L. Lawler: The Great Mathematical Sputnik of 1979, *Math. Intelligencer* 2(1980) 191–198,

which is a remarkable article about the history of Khachyan’s result, is not only of historical interest:

The *Times* story appears to have been based on certain unshakable preconceptions of its writer, Malcolm W. Browne. Browne called George Dantzig, of Stanford University, a great pioneering authority on linear programming, and tried to force him into various admissions. Dantzig’s version of the interview bears repeating:

“What about the traveling salesman problem?” asked Browne. “If there is a connection, I don’t know what it is,” said Dantzig. (“The Russian discovery proposed an approach for [solving] a class of problems related to the “Traveling Salesman Problem,” reported Browne.) “What about cryptography?” asked Browne. “If there is a connection, I don’t know what it is,” said Dantzig. (“The theory of codes could eventually be affected,” reported Browne.) “Is the Russian method practical?” asked Browne. “No,” said Dantzig. (“Mathematicians describe the discovery . . . as a method by which computers can find solutions to a class of very hard problems that has hitherto been attacked on a hit-or-miss basis,” reported Browne.)

and similarly, $\langle A \rangle = \sum_{i=1}^m \sum_{j=1}^n \langle a_{ij} \rangle$ for a rational $m \times n$ matrix A . If we consider a linear program L , say in the form

$$\text{maximize } \mathbf{c}^T \mathbf{x} \text{ subject to } A\mathbf{x} \leq \mathbf{b},$$

and if we restrict ourselves to the case of A , $\mathbf{b}$, and $\mathbf{c}$ rational (which is a reasonable assumption from a computational perspective), then the bit size of L is $\langle L \rangle = \langle A \rangle + \langle \mathbf{b} \rangle + \langle \mathbf{c} \rangle$.

We say that an algorithm is a **polynomial algorithm for linear programming** if a polynomial $p(x)$ exists such that for every linear program L with rational A , $\mathbf{b}$, and $\mathbf{c}$ the algorithm finds a correct solution in at most $p(\langle L \rangle)$ steps. The steps are counted in some of the usual models of computation, for example, as steps of a Turing machine (usually the chosen computational model is not crucial; whatever is polynomial in one model is also polynomial in other reasonable models). We stress right away that a single arithmetic operation is not counted as a single step here! We count as steps operations with single bits, and hence, addition of two k -bit integers requires at least k steps.

Let us digress briefly from linear programming and let us consider Gaussian elimination, a well-known algorithm for solving systems of linear equations. For a system $A\mathbf{x} = \mathbf{b}$, where (for simplicity) A is an $n \times n$ matrix and both A and $\mathbf{b}$ are rational, we naturally define the input size as $\langle A \rangle + \langle \mathbf{b} \rangle$. Is Gaussian elimination a polynomial algorithm? This is a tricky question! Although this algorithm needs only order of n^3 arithmetic operations, the catch is that too large intermediate values could come up during the computation, even if all entries in A and in $\mathbf{b}$ are small integers. If, for example, integers with as many as 2^n bits ensued, which can indeed happen in a naive implementation of Gaussian elimination, the computation would need exponentially many steps, although it would involve only n^3 arithmetic operations. (All of this concerns *exact* computations, while many implementations use floating-point arithmetic and hence the numbers are continually rounded. But then there is no guarantee that the results are correct.) We do not want to scare the reader needlessly: It is known how Gaussian elimination can be implemented in polynomial time. We want only to point out that this is not self-evident (and not too simple, either), and call attention to one kind of trouble that may develop in attempts at proving polynomiality.

The ellipsoid method, as well as some of the interior point methods, are polynomial, while the simplex method with Bland's rule (and with many other pivot rules too) is not polynomial.²

² The nonpolynomiality is proved by means of the Klee–Minty cube; see Section 5.9. One has to check that an n -dimensional Klee–Minty cube can be represented by input of size polynomial in n .

Strongly polynomial algorithms. For algorithms whose input is described by a sequence of integers or rationals, such as algorithms for linear programming, the number of arithmetic operations (addition, subtraction, multiplication, division, exponentiation) is also considered, together with the number of bit operations. This often gives a more realistic picture of the running time, because contemporary computers usually execute an arithmetic operation as an elementary step, provided that the operands are not too large.

A suitable implementation of Gaussian elimination is, on the one hand, a polynomial algorithm in the sense discussed above, and on the other hand, the number of arithmetic operations is bounded by a polynomial, namely by the polynomial Cn^3 for a suitable constant C , where n is the number of equations in the system and also the number of variables. The number of arithmetic operations thus depends only on n , and it is the same for input numbers with 10 bits as for input numbers with a million bits. We say that Gaussian elimination is a **strongly polynomial** algorithm for solving systems of linear equations.

A strongly polynomial algorithm for linear programming would be one that, first, would be polynomial in the sense defined above, and second, for every linear program with n variables and m constraints it would find a solution using at most $p(m+n)$ arithmetic operations, where $p(x)$ is a fixed polynomial. But no strongly polynomial algorithm for linear programming is known, and finding one is a major open problem.

The ellipsoid method is not strongly polynomial. For every natural number M one can find a linear program with only 2 variables and 2 constraints for which the ellipsoid method executes at least M arithmetic operations (the coefficients in such linear programs must have bit size tending to infinity as $M \rightarrow \infty$). In particular, the number of arithmetic operations for the ellipsoid method cannot be bounded by any polynomial in $m+n$.

Ellipsoids. A two-dimensional ellipsoid is an ellipse plus its interior. An ellipsoid in general can most naturally be introduced as an affine transformation of a ball. We let

$$B^n = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{x}^T \mathbf{x} \leq 1\}$$

be the n -dimensional ball of unit radius centered at $\mathbf{0}$. Then an n -dimensional **ellipsoid** is a set of the form

$$E = \{M\mathbf{x} + \mathbf{s} : \mathbf{x} \in B^n\},$$

where M is a nonsingular $n \times n$ matrix and $\mathbf{s} \in \mathbb{R}^n$ is a vector. The mapping $\mathbf{x} \mapsto M\mathbf{x} + \mathbf{s}$ is a composition of a linear function and a translation; this is called an **affine map**.

By manipulating the definition we can describe the ellipsoid by an inequality:

$$\begin{aligned} E &= \{\mathbf{y} \in \mathbb{R}^n : M^{-1}(\mathbf{y} - \mathbf{s}) \in B^n\} \\ &= \{\mathbf{y} \in \mathbb{R}^n : (\mathbf{y} - \mathbf{s})^T (M^{-1})^T M^{-1} (\mathbf{y} - \mathbf{s}) \leq 1\} \\ &= \{\mathbf{y} \in \mathbb{R}^n : (\mathbf{y} - \mathbf{s})^T Q^{-1} (\mathbf{y} - \mathbf{s}) \leq 1\}, \end{aligned} \quad (7.1)$$

where we have set $Q = MM^T$. It is well known and easy to check that such a Q is a positive definite matrix, that is, a symmetric square matrix satisfying $\mathbf{x}^T Q \mathbf{x} > 0$ for all nonzero vectors $\mathbf{x}$. Conversely, from matrix theory it is known that each positive definite matrix Q can be factored as $Q = MM^T$ for some nonsingular square matrix M . Therefore, an equivalent definition is that an ellipsoid is a set described by (7.1) for some positive definite Q and some $\mathbf{s}$.

Geometrically, $\mathbf{s}$ is the center of the ellipsoid E . If Q is a diagonal matrix and $\mathbf{s} = \mathbf{0}$, then we have an ellipsoid in *axial position*, of the form

$$\left\{ \mathbf{y} \in \mathbb{R}^n : \frac{y_1^2}{q_{11}} + \frac{y_2^2}{q_{22}} + \cdots + \frac{y_n^2}{q_{nn}} \leq 1 \right\}.$$

The axes of this ellipsoid are parallel to the coordinate axes. The numbers $\sqrt{q_{11}}, \sqrt{q_{22}}, \dots, \sqrt{q_{nn}}$ are the lengths of the *semiaxes* of the ellipsoid E , which may sound familiar to those accustomed to the equation of an ellipse of the form $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$. As is taught in linear algebra in connection with eigenvalues, each positive definite matrix Q can be diagonalized by an orthogonal basis change. That is, there exists an orthogonal matrix T such that the matrix TQT^{-1} is diagonal, with the eigenvalues of Q on the diagonal. Geometrically, T represents a rotation of the coordinate system that brings the ellipsoid into axial position.

A lion in the Sahara. A traditional mathematical anecdote gives directions for hunting a lion in the Sahara (under the assumption that there is at most one). We fence all of the Sahara, we divide it into two halves by another fence, and we detect one half that has no lion in it. Then we divide the other half by a fence, and we continue in this manner until the fenced piece of ground is so small that the lion cannot move and so it is caught, or if there is no lion in it, we have proved that there was none in the Sahara either. Although the qualities of this hunting guide can be disputed, for us it is essential that it gives a reasonably good description of the ellipsoid method. But in the real ellipsoid method we insist that the currently fenced piece is always an ellipsoid, even at the price that the lion can sometimes return to places from where it was expelled earlier; it is only guaranteed that the area of its territory shrinks all the time.

The ellipsoid method doesn't directly solve a linear program, but rather it seeks a solution of a system of linear inequalities $A\mathbf{x} \leq \mathbf{b}$. But as we know,

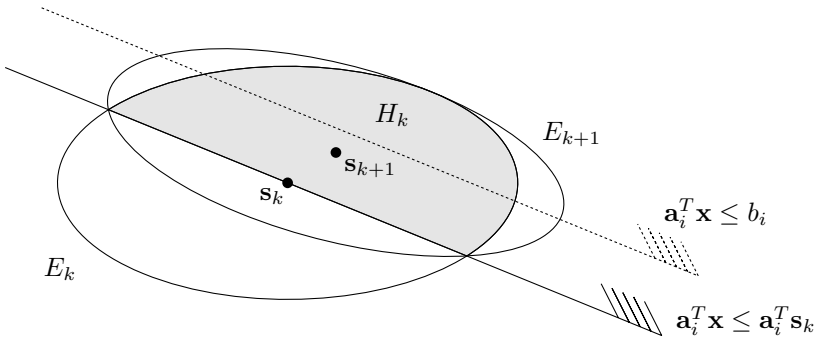
this is sufficient for solving a linear program (see Section 6.1). For a simpler exposition we will first consider the following softened version of the problem:

Together with the matrix A and vector $\mathbf{b}$ we are given rational numbers $R > \varepsilon > 0$. We assume that the set $P = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} \leq \mathbf{b}\}$ is contained in the ball $B(\mathbf{0}, R)$ centered at $\mathbf{0}$ with radius R . If P contains a ball of radius ε , then the algorithm has to return a point $\mathbf{y} \in P$. However, if P contains no ball of radius ε , then the algorithm may return either some $\mathbf{y} \in P$, or the answer **NO SOLUTION**.

The ball $B(\mathbf{0}, R)$ thus plays the role of the Sahara and we assume that the lion, if present, is at least ε large. If there is only a smaller lion in the Sahara, it may escape or we may catch it—we don't care.

Under these assumptions, the ellipsoid method generates a sequence of ellipsoids $E_0, E_1, \dots, E_t$, where $P \subseteq E_k$ for each k , as follows:

1. Set $k = 0$ and $E_0 = B(\mathbf{0}, R)$.
2. Let the current ellipsoid E_k be of the form $E_k = \{\mathbf{y} \in \mathbb{R}^n : (\mathbf{y} - \mathbf{s}_k)^T Q_k^{-1} (\mathbf{y} - \mathbf{s}_k) \leq 1\}$. If $\mathbf{s}_k$ satisfies all inequalities of the system $A\mathbf{x} \leq \mathbf{b}$, return $\mathbf{s}_k$ as a solution; **stop**.
3. Otherwise, choose an inequality of the system that is violated by $\mathbf{s}_k$. Let it be the i th inequality; so we have $\mathbf{a}_i^T \mathbf{s}_k > b_i$. Define E_{k+1} as the ellipsoid of the smallest possible volume containing the “half-ellipsoid” $H_k = E_k \cap \{\mathbf{x} \in \mathbb{R}^n : \mathbf{a}_i^T \mathbf{x} \leq \mathbf{a}_i^T \mathbf{s}_k\}$; see the following picture:



4. If the volume of E_{k+1} is smaller than the volume of a ball of radius ε , return **NO SOLUTION**; **stop**. Otherwise, increase k by 1 and continue with Step 2.

Let H'_k denote the intersection of the ellipsoid E_k with the half-space $\{x \in \mathbb{R}^n : \mathbf{a}_i^T \mathbf{x} \leq b_i\}$ defined by the i th inequality of the system. If $P \subseteq E_k$,

then also $P \subseteq H'_k$, and the more so $P \subseteq H_k$. Why is the smallest ellipsoid containing H_k taken for E_{k+1} , instead of the smallest ellipsoid containing H'_k ? Purely in order to simplify the analysis of the algorithm, since the equation of E_{k+1} comes out less complicated this way.

The ellipsoid E_{k+1} , i.e., the ellipsoid of the smallest volume containing H_k , is always determined uniquely. For illustration we mention that it is given by

$$\begin{aligned} \mathbf{s}_{k+1} &= \mathbf{s}_k - \frac{1}{n+1} \cdot \frac{Q\mathbf{s}_k}{\sqrt{\mathbf{s}_k^T Q \mathbf{s}_k}}, \\ Q_{k+1} &= \frac{n^2}{n^2-1} \left(Q_k - \frac{2}{n+1} \cdot \frac{Q\mathbf{s}_k\mathbf{s}_k^T Q}{\mathbf{s}_k^T Q \mathbf{s}_k} \right). \end{aligned}$$

We also leave without proof a fact crucial for the proof of correctness and efficiency of the ellipsoid method: We always have

$$\frac{\text{volume}(E_{k+1})}{\text{volume}(E_k)} \leq e^{-1/(2n+2)}.$$

Hence the volume of the ellipsoid E_k is at least $e^{k/(2n+2)}$ times smaller than the volume of the initial ball $B(\mathbf{0}, R)$. Since the volume of an n -dimensional ball is proportional to the n th power of the radius, for k satisfying $R \cdot e^{-k/n(2n+2)} < \varepsilon$ the volume of E_k is smaller than that of a ball of radius ε . Such k provides an upper bound of $\lceil n(2n+2) \ln(R/\varepsilon) \rceil$ on the maximum number of iterations. This is bounded by a polynomial in $n + \langle R \rangle + \langle \varepsilon \rangle$.

So much for the simple and beautiful idea of the ellipsoid method—now we are coming to manageable but unpleasant complications. First of all, we cannot compute the ellipsoid E_{k+1} exactly, at least not in rational arithmetic, since the defining formulas contain square roots. To get around this, E_{k+1} is computed only approximately with a suitable precision. But one has to be careful so that P is still guaranteed to be contained in E_{k+1} , and thus the approximate E_{k+1} has to be expanded slightly.

Another trouble arises, for example, when the same inequality is used for cutting the current ellipsoid in many iterations in a row. Then the ellipsoids may become too long (needle-like), and they have to be shortened artificially.

Yet another problem is that we don't really want to solve the softened problem with R and ε , but an arbitrary system of linear inequalities without any restrictions. Here the bound on the bit size of the entries of A and $\mathbf{b}$ comes into play, through the following facts:

- (E1) (If a solution exists, then there is a not too large solution.) Let $\varphi = \langle A \rangle + \langle \mathbf{b} \rangle$ denote the input size for the system $A\mathbf{x} \leq \mathbf{b}$. Then

this system has a solution if and only if the system

$$\begin{aligned} A\mathbf{x} &\leq \mathbf{b} \\ -K &\leq x_1 \leq K \\ -K &\leq x_2 \leq K \\ &\vdots \\ -K &\leq x_n \leq K \end{aligned}$$

has a solution, where $K = 2^\varphi$. Clearly, all solutions of the latter system are contained in the ball $B(\mathbf{0}, R)$, where $R = K\sqrt{n}$.

- (E2) (If a solution exists, then the solution set of a slightly relaxed system contains a small ball.) Let us put $\eta = 2^{-5\varphi}$, $\varepsilon = 2^{-6\varphi}$, and let $\boldsymbol{\eta}$ be the n -component vector with all components equal to η . Then the system $A\mathbf{x} \leq \mathbf{b}$ has a solution if and only if the system $A\mathbf{x} \leq \mathbf{b} + \boldsymbol{\eta}$ has a solution, and in such case the solution set of the latter system contains a ball of radius ε .

It is not hard to see how these facts can be used for solving an arbitrary system $A\mathbf{x} \leq \mathbf{b}$ by the ellipsoid method. Instead of this system we solve the softened problem by the ellipsoid method, but for a new system $A\mathbf{x} \leq \mathbf{b} + \boldsymbol{\eta}$, $-K - \eta \leq x_1 \leq K + \eta$, $-K - \eta \leq x_2 \leq K + \eta$, $\dots$, $-K - \eta \leq x_n \leq K + \eta$, where K , R , ε , and η are chosen suitably (first we add the constraints $-K \leq x_i \leq K$ as in (E1), and then we apply (E2) to the resulting system). It is important that the bit size of R and ε , as well as the input size of the new system, are bounded by a polynomial function of φ . Thus the ellipsoid method runs in polynomial time, and it always finds a solution of $A\mathbf{x} \leq \mathbf{b}$ if it exists.

We will not prove facts (E1) and (E2) here, but we sketch the basic ideas. For (E1) we first discuss the case $n = 2$ (in the plane). Let us consider a system of m inequalities

$$a_{i1}x + a_{i2}y \leq b_i, \quad i = 1, 2, \dots, m.$$

Let ℓ_i be the line $\{(x, y) \in \mathbb{R}^2 : a_{i1}x + a_{i2}y = b_i\}$. It is easy to calculate that the intersection of ℓ_i and ℓ_j , if it exists, has coordinates

$$\left(\frac{a_{i2}b_j - a_{j2}b_i}{a_{i2}a_{j1} - a_{i1}a_{j2}}, \frac{a_{j1}b_i - a_{i1}b_j}{a_{i2}a_{j1} - a_{i1}a_{j2}} \right).$$

If, for example, all a_{ij} and b_i are integers with absolute value at most 1000, then the coordinates of all intersections are fractions with numerators and denominators bounded by $2 \cdot 10^6$ in absolute value. Thus, if the solution set of the considered system of inequalities has at least one vertex, such a vertex has to lie in the square $[-2 \cdot 10^6, 2 \cdot 10^6]^2$. If the solution set has no vertex and it is nonempty, it can be shown that it has to contain one of the lines ℓ_i , and that each ℓ_i intersects

the just-mentioned square. Fact (E1) can be verified along these lines for the considered system with two variables. For a general system in dimension n the idea is the same, and Cramer's rule and a bound on the magnitude of the determinant of a matrix are used for estimating the coordinates of vertices of the solution set.

Fact (E2) requires more work, but the idea is similar. Each solution $\tilde{\mathbf{x}}$ of the original system $A\mathbf{x} \leq \mathbf{b}$ also satisfies the modified system $A\mathbf{x} \leq \mathbf{b} + \boldsymbol{\eta}$, and all $\mathbf{x}$ from the ball $B(\tilde{\mathbf{x}}, \varepsilon)$ satisfy it as well, because changing $\mathbf{x}$ by ε cannot change any coordinate of $A\mathbf{x}$ by more than η .

If $A\mathbf{x} \leq \mathbf{b}$ has no solution, then by a suitable variant of the Farkas lemma, namely, Proposition 6.4.3(iii), there exists a nonnegative $\mathbf{y} \in \mathbb{R}^m$ such that $\mathbf{y}^T A = \mathbf{0}^T$ and $\mathbf{y}^T \mathbf{b} < 0$, and by normalizing $\mathbf{y}$ we may assume $\mathbf{y}^T \mathbf{b} = -1$. By Cramer's rule again it is shown that there also exists a $\mathbf{y}$ with not too large components, and such $\mathbf{y}$ then witnesses unsolvability for the system $A\mathbf{x} \leq \mathbf{b} + \boldsymbol{\eta}$ as well.

Here we finish the outline of the ellipsoid method. If some parts were too incomplete and hazy for the reader, we can only recommend a more extensive treatment, for instance in the excellent book

M. Grötschel, L. Lovász, L. Schrijver: *Geometric Algorithms and Combinatorial Optimization*, 2nd edition, Springer, Heidelberg 1994.

(We have taken the Sahara metaphor from there, among others.)

Why ellipsoids? They are used in the ellipsoid method since they constitute probably the simplest class of n -dimensional convex sets that is closed under nonsingular affine maps. Popularly speaking, this class is rich enough to approximate all convex polyhedra including flat ones and needle-like ones. If desired, ellipsoids can be replaced by simplices, for example, but the formulas in the algorithm and its analysis become considerably more unpleasant than those for ellipsoids.

The ellipsoid method need not know all of the linear program. The system of inequalities $A\mathbf{x} \leq \mathbf{b}$ can also be given by means of a **separation oracle**. This is an algorithm (black box) that accepts a point $\mathbf{s} \in \mathbb{R}^n$ as input, and if $\mathbf{s}$ is a solution of the system, it returns the answer YES, while if $\mathbf{s}$ is not a solution, it returns one (arbitrary) inequality of the system that is violated by $\mathbf{s}$. (Such an inequality separates $\mathbf{s}$ from the solution set, and hence the name separation oracle.) The ellipsoid method calls the separation oracle with the centers $\mathbf{s}_k$ of the generated ellipsoids, and it always uses the violated inequality returned by the oracle for determining the next ellipsoid.

We talk about this since a separation oracle can be implemented efficiently for some interesting optimization problems even when the full system has exponentially many inequalities or even infinitely many (so far we haven't considered infinite systems at all).

Probably the most important example of a situation in which an infinite system of linear inequalities can be solved by the ellipsoid method is **semidefinite programming**. In a semidefinite program we consider not an unknown *vector* $\mathbf{x}$, but rather an unknown square *matrix* $X = (x_{ij})_{i,j=1}^n$. We optimize a linear function of the variables x_{ij} . The constraints are linear inequalities and equations for the x_{ij} , *plus* the requirement that the matrix X has to be *positive semidefinite*. The last constraint distinguishes semidefinite programming from linear programming. It can be expressed by a system of infinitely many linear inequalities, namely, $\mathbf{a}^T X \mathbf{a} \geq 0$ for each $\mathbf{a} \in \mathbb{R}^n$. A separation oracle for this system can be constructed based on algorithms for computing eigenvalues and eigenvectors. The ellipsoid method can then approximate the optimum with a prescribed precision in polynomial time. (In reality, though, things are not quite as simple as it might seem from our mini-description.)

Numerous computational problems can be solved in polynomial time via semidefinite programming, some of them exactly and some approximately, and sometimes this yields the only known polynomial algorithm. A nice example of application of semidefinite programming is the **maximum cut problem** (MAXCUT), in which the vertex set of a given graph $G = (V, E)$ should be divided into two parts so that the maximum possible number of edges go between the parts. Semidefinite programming is an essential component of an approximation algorithm for MAXCUT, called the Goemans–Williamson algorithm, that always computes a partition with the number of edges going between the parts at least 87.8% of the optimal number. This is the best known approximation guarantee, and most likely also the best possible one for any polynomial algorithm. More about this and related topics can be found, for instance, in the survey

L. Lovász: Semidefinite programs and combinatorial optimization, in *Recent Advances in Algorithms and Combinatorics* (B. Reed and C. Linhares-Sales, editors), pages 137–194, Springer, New York, 2003.

Let us remark that in the just outlined applications, the ellipsoid method can be replaced by certain interior point methods (the so-called volumetric-center methods, which are not mentioned in our brief discussion of interior point methods in Section 7.2 below), and this yields algorithms efficient both in theory and in practice. See

K. Krishnan, T. Terlaky: Interior point and semidefinite approaches in combinatorial optimization, in: D. Avis, A. Hertz, and O. Marcotte (editors): *Graph Theory and Combinatorial Optimization*, Springer, Berlin etc. 2005, pages 101–158.

Theory versus practice. The notion of polynomial algorithm was suggested in the 1970s by Jack Edmonds as a formalized counterpart of the intuitive notion of an efficient algorithm. Today, a theoretician's first question for every algorithm is, Is it polynomial?

How is it possible that the ellipsoid method, which is polynomial, is much slower in practice than the simplex method, which is not polynomial? One of the reasons is that even though the ellipsoid method is polynomial, the degree of the polynomial is quite high. The second and main reason is that the simplex method is slow only on artificially constructed linear programs, which it almost never encounters in practice, while the ellipsoid method seldom behaves better than in the worst case. But the “good behavior on all inputs with rare exceptions” of the simplex method seems hard to capture theoretically. Moreover, a guaranteed efficiency for *all* inputs is much more satisfactory than only an empirically supported belief that an algorithm is usually fast.

The notion of polynomial algorithm thus has great shortcomings from a practical point of view. But attempts at constructing a polynomial algorithm in theory usually also leads, over time, to practically efficient algorithms. An impressive example in the area of linear programming are interior point methods.

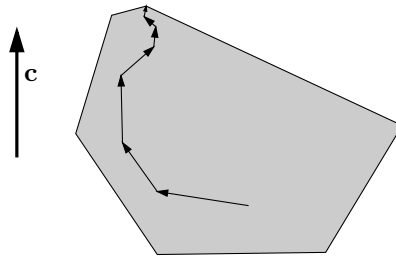
7.2 Interior Point Methods

The next time linear programming made it to press headlines was in 1984. Narendra Karmakar, a researcher at IBM, suggested an algorithm that is named after him and belongs to the large family of interior point methods. He proved its polynomiality and published results of computational experiments suggesting that in practice it is much faster than the simplex method. Although his statements in the latter direction turned out to be somewhat exaggerated, interior point methods are nowadays commonly used in linear programming and often they beat the simplex method, especially on very large linear programs. They are also applied with success to semidefinite programming and other important classes of optimization problems, such as convex quadratic programming.

Interior point methods have been used for nonlinear optimization problems at least since the 1950s. For linear programs they were actually tested by the early 1970s, and interestingly, none was found competitive to the simplex method. This was because theory and hardware were not advanced enough—indeed, interior point methods typically outperform the simplex methods only on problems so large that they were beyond the capabilities of the computers at that time, and moreover, efficient implementation of interior point methods relies on powerful routines for solving large but sparse systems of linear

equations, which were not available either. The success story began only with Karmakar's results.

The basic approach. When solving a linear program, the simplex method crawls along the boundary of the set of feasible solutions. The ellipsoid method encircles the set of feasible solutions, and up until the last step it remains outside of it. Interior point methods walk through the interior of the set of feasible solutions toward an optimum, carefully avoiding the boundary. Only at the very end, when they get almost to an optimum, they jump to an exact optimum by a rounding step. See the following schematic picture:



Working with an interior point all the time is the key idea that gave the methods their name. Interior points possess various pleasant mathematical properties, a kind of “nondegeneracy,” and they allow one to avoid intricacies of the combinatorial structure of the boundary, which have been haunting research of the simplex method for decades. The art of interior point methods is how to keep away from the boundary while still progressing to the optimum. (For the sake of exposition, let us now assume that we are dealing with a linear program in which some initial interior point is available.)

There are several rather different basic approaches in interior point methods, and each has many variants. Interior point methods in linear programming are classified as *central path methods* (or *central trajectory methods*), *potential reduction methods*, and *affine scaling methods*, and for almost every approach one can consider a *primal version*, a *dual version*, a *primal-dual version*, or a *self-dual version*.

Here we will consider only central path methods, which have been computationally the most successful. We will present a single algorithm from this family: one with the best known theoretical complexity bounds and very good practical performance. It is fair to say, though, that a number of different interior point methods yield the same theoretical bounds and that many practitioners might say that, from their point of view, there are even better algorithms.

The analysis of the algorithm, as well as some important details of its implementation, are somewhat complicated and we will not discuss them. Modern linear programming software is based on quite advanced mathematics; for example, as was remarked above, one of the keys to the success of

interior point methods is an efficient solver of special systems of linear equations, much more sophisticated than good old Gaussian elimination.

In addition to the existence of innumerable variants of interior point methods in the literature, different expositions of the same algorithm may be based on different intuition. For example, what is presented as a step of Newton's iteration method in one source may be derived by linear approximation in another source. We say this so that a reader who finds in the literature something seemingly rather different from what we say below is not confused more than necessary.

The central path. First we explain the (mathematical) notion of central path. To this end, we consider an arbitrary convex polyhedron P in $\mathbb{R}^n$ defined by a system $A\mathbf{x} \leq \mathbf{b}$ of m linear inequalities, and a linear objective function $f(\mathbf{x}) = \mathbf{c}^T \mathbf{x}$ as usual. We introduce a family of auxiliary objective functions f_μ , depending on a parameter $\mu \in [0, \infty)$:

$$f_\mu(\mathbf{x}) = \mathbf{c}^T \mathbf{x} + \mu \cdot \sum_{i=1}^m \ln(b_i - \mathbf{a}_i \mathbf{x}),$$

where $\mathbf{a}_i$ is the i th row of the matrix A (regarded here as a row vector). Thus $f_0 = f$ is the original linear objective function, while the f_μ for $\mu > 0$ are nonlinear, due to the logarithms. The function f_μ is constructed in such a way that when $\mathbf{x}$ approaches the boundary of P , i.e., the difference of the right-hand side and left-hand side of some inequality approaches 0, then f_μ tends to $-\infty$. The expression following μ in the definition of f_μ is called a *barrier function*, or more definitely, a *logarithmic barrier*. The word barrier is more fitting for a minimization problem, in our case minimizing $-f_\mu$, where the graph of the objective function has barriers preventing minimization algorithms from hitting the walls.

We thus consider the auxiliary problem of maximizing $f_\mu(\mathbf{x})$ over P , for given $\mu > 0$. Since f_μ is undefined on the boundary of P , we actually maximize over $\text{int}(P)$, the interior of P , and so for the problem to make sense, we need to assume that $\text{int}(P) \neq \emptyset$.

If we assume that, moreover, P is bounded, then f_μ attains a maximum at a unique point in the interior of P , which we denote by $\mathbf{x}^*(\mu)$.

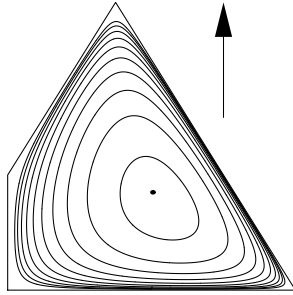
Indeed, the existence of a maximum follows from the well-known fact that every continuous function attains a maximum on a compact set: The appropriate compact set is $\{\mathbf{x} \in \text{int}(P) : f_\mu(\mathbf{x}) \geq f_\mu(\mathbf{x}_0)\}$, where $\mathbf{x}_0 \in \text{int}(P)$ is arbitrary (a little argument, which we leave to the reader, is needed to verify that this set is closed).

As for uniqueness, let us assume that f_μ attains a maximum at two distinct feasible points $\mathbf{x}$ and $\mathbf{y}$. Then $f_\mu(\mathbf{x}) = f_\mu(\mathbf{y})$, and since f_μ is easily seen to be concave (meaning that $-f_\mu$ is convex), it follows

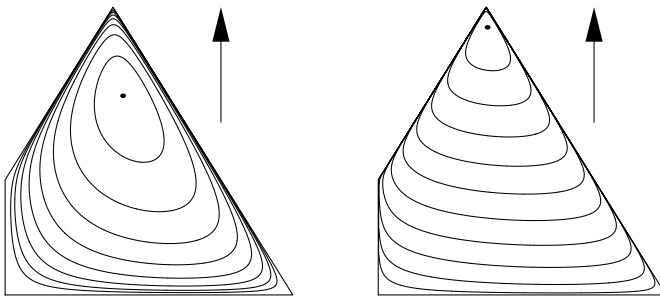
that f_μ has to be constant on the segment $\mathbf{x}\mathbf{y}$. Since the logarithm is strictly concave, this can happen only if $A\mathbf{x} = A\mathbf{y}$. But then P would contain all of the line $\{\mathbf{x} + s(\mathbf{y} - \mathbf{x}) : s \in \mathbb{R}\}$ and would not be bounded—a contradiction. In short, the maximum is unique because f_μ is strictly concave.

The condition of P bounded can be relaxed somewhat, but some condition forcing f_μ to be bounded above on P is clearly necessary, as is documented by $P = \{x \in \mathbb{R}^1 : x_1 \geq 0\}$ and either $\mathbf{c} = (1)$ or $\mathbf{c} = (0)$.

If μ is a very large number, the influence of the term $\mathbf{c}^T \mathbf{x}$ in f_μ is negligible and $\mathbf{x}^*(\mu)$ is a point “farthest from the boundary,” called the *analytic center* of P . The following picture shows, for a two-dimensional P , contour lines of the function f_μ for $\mu = 100$ (the vector $\mathbf{c}$ is depicted by an arrow and the point $\mathbf{x}^*(\mu)$ by a dot):³



On the other hand, for small μ the point $\mathbf{x}^*(\mu)$ is close to an optimum of $\mathbf{c}^T \mathbf{x}$; see the illustrations below for $\mu = 0.5$ and $\mu = 0.1$:



The **central path** is defined as the set $\{\mathbf{x}^*(\mu) : \mu > 0\}$. We stress that the central path is not associated with P itself, but rather with a particular system of inequalities defining P and a particular linear objective function $\mathbf{c}^T \mathbf{x}$.

³ A contour line of a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ is a set of the form $f^{-1}(\{\alpha\})$, $\alpha \in \mathbb{R}$.

The idea of central path methods is to start at $\mathbf{x}^*(\mu)$ with a suitable large μ , and then follow the central path, decreasing μ until an optimum of $\mathbf{c}^T \mathbf{x}$ is reached. Computing $\mathbf{x}^*(\mu)$ exactly would be difficult, and so actual algorithms follow the central path only approximately.

A linear program in equational form and the primal–dual central path. Having introduced the concept of central path in a geometrically transparent way, we change the setting slightly. It turns out that for the actual algorithm it is better to replace general inequality constraints by equations and nonnegativity constraints; this is similar to the case of the simplex method. So now we consider the usual linear program in equational form

$$\text{maximize } \mathbf{c}^T \mathbf{x} \text{ subject to } A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq \mathbf{0}, \quad (7.2)$$

where A is an $m \times n$ matrix of rank m . Here the barrier function should prevent violation of the nonnegativity constraints (while the equations only restrict everything to a subspace of $\mathbb{R}^n$ and they do not enter the objective function). So we set

$$f_\mu(\mathbf{x}) = \mathbf{c}^T \mathbf{x} + \mu \cdot \sum_{j=1}^n \ln x_j$$

and consider the auxiliary problem

$$\text{maximize } f_\mu(\mathbf{x}) \text{ subject to } A\mathbf{x} = \mathbf{b}, \mathbf{x} > \mathbf{0},$$

where the notation $\mathbf{x} > \mathbf{0}$ means that *all* coordinates of $\mathbf{x}$ are strictly positive.

We would again like to claim that under suitable conditions, the auxiliary problem has a unique maximizer $\mathbf{x}^*(\mu)$ for every $\mu > 0$. Obviously, we need to assume that there is a feasible $\mathbf{x} > \mathbf{0}$. Also, once we make sure that f_μ attains *at least one* maximum, it is easily seen, as above, that the maximum is unique.

We now derive *necessary* conditions for the existence of a maximum, and at the same time, we express $\mathbf{x}^*(\mu)$ as a solution of a system of equations. Later on, we will check that the necessary conditions are also sufficient. We derive these conditions by the method of **Lagrange multipliers** from analysis.

We recall that this is a general method for maximization of $f(\mathbf{x})$ subject to m constraints $g_1(\mathbf{x}) = 0, g_2(\mathbf{x}) = 0, \dots, g_m(\mathbf{x}) = 0$, where f and $g_1, \dots, g_m$ are functions from $\mathbb{R}^n$ to $\mathbb{R}$. It can be seen as a generalization of the basic calculus trick for maximizing a univariate function by seeking a zero of its derivative. It introduces the following system of equations with unknowns $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^m$ (the y_i are auxiliary variables called the *Lagrange multipliers*):

$$g_1(\mathbf{x}) = g_2(\mathbf{x}) = \dots = g_m(\mathbf{x}) = 0 \quad \text{and} \quad \nabla f(\mathbf{x}) = \sum_{i=1}^m y_i \nabla g_i(\mathbf{x}). \quad (7.3)$$

Here ∇ denotes the gradient (which by convention is a *row* vector):

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f(\mathbf{x})}{\partial x_1}, \frac{\partial f(\mathbf{x})}{\partial x_2}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right).$$

That is, ∇f is a vector function from $\mathbb{R}^n$ to $\mathbb{R}^n$ whose i th component is the partial derivative of f with respect to x_i . Thus, the equation $\nabla f(\mathbf{x}) = \sum_{i=1}^m y_i \nabla g_i(\mathbf{x})$ stipulates the equality of two n -component vectors. The method of Lagrange multipliers tells us that a maximum of $f(\mathbf{x})$ subject to $g_1(\mathbf{x}) = g_2(\mathbf{x}) = \dots = g_m(\mathbf{x}) = 0$ occurs at $\mathbf{x}$ satisfying (7.3); that is, there exists $\mathbf{y}$ such that the considered $\mathbf{x}$ and this $\mathbf{y}$ together fulfill (7.3) (a special case of this result is derived in Section 8.7). Of course, we need some conditions on f and the g_i . It suffices to require that f and the g_i be defined on a nonempty open subset of $\mathbb{R}^n$ and have continuous first partial derivatives there, and this will be obviously satisfied in our simple application.

We apply the method of Lagrange multipliers to maximizing $f_\mu(\mathbf{x})$ subject to $A\mathbf{x} = \mathbf{b}$ (the nonnegativity constraints are taken care of implicitly, by the barriers). So we set $g_i(\mathbf{x}) = b_i - \mathbf{a}_i\mathbf{x}$. Then, after a little manipulation, the system(7.3) becomes

$$A\mathbf{x} = \mathbf{b}, \quad \mathbf{c} + \mu \left(\frac{1}{x_1}, \frac{1}{x_2}, \dots, \frac{1}{x_n} \right) = A^T \mathbf{y}.$$

A more convenient form of this system is obtained by introducing an auxiliary nonnegative vector $\mathbf{s} = \mu \cdot \left(\frac{1}{x_1}, \frac{1}{x_2}, \dots, \frac{1}{x_n} \right) \in \mathbb{R}^n$. We rewrite the relation of $\mathbf{s}$ and $\mathbf{x}$ to $(s_1x_1, s_2x_2, \dots, s_nx_n) = \mu\mathbf{1}$, with $\mathbf{1}$ denoting the vector of all 1's. Then $\mathbf{x}^*(\mu)$ is expressed as the $\mathbf{x}$ -part of a solution of the following system with unknowns $\mathbf{x}, \mathbf{s} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^m$:

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ A^T \mathbf{y} - \mathbf{s} &= \mathbf{c} \\ (s_1x_1, s_2x_2, \dots, s_nx_n) &= \mu\mathbf{1} \\ \mathbf{x}, \mathbf{s} &\geq \mathbf{0}. \end{aligned} \tag{7.4}$$

All of these equations are linear except for $(s_1x_1, s_2x_2, \dots, s_nx_n) = \mu\mathbf{1}$.

Although we have derived the system (7.4) assuming $\mu > 0$, let us make a small digression and look at what (7.4) tells us for $\mu = 0$. Then, for nonnegative $\mathbf{x}$ and $\mathbf{s}$, the equation $(s_1x_1, s_2x_2, \dots, s_nx_n) = \mathbf{0}$ is equivalent to $\mathbf{s}^T \mathbf{x} = 0$, and since $\mathbf{s} = A^T \mathbf{y} - \mathbf{c}$, we have $0 = \mathbf{s}^T \mathbf{x} = \mathbf{y}^T A\mathbf{x} - \mathbf{c}^T \mathbf{x} = \mathbf{y}^T \mathbf{b} - \mathbf{c}^T \mathbf{x}$ (using $A\mathbf{x} = \mathbf{b}$). This may remind one of the equality of objective functions for the primal and dual linear programs. And indeed, if we take (7.2) as a primal linear program, the dual is

$$\text{minimize } \mathbf{b}^T \mathbf{y} \text{ subject to } A^T \mathbf{y} \geq \mathbf{c}, \mathbf{y} \in \mathbb{R}^m. \tag{7.5}$$

Then (7.4) for $\mu = 0$ tells us exactly that $\mathbf{x}$ is a feasible solution of the primal linear program (7.2), $\mathbf{y}$ is a feasible solution of the dual linear program (7.5) (here the s_j serve as slack variables expressing the difference $A^T \mathbf{y} - \mathbf{c}$), and the objective functions are equal! Hence such $\mathbf{x}$ and $\mathbf{y}$ are optimal.

So far we have shown that *if* the problem of maximizing $f_\mu(\mathbf{x})$ subject to $A\mathbf{x} = \mathbf{b}$ and $\mathbf{x} > \mathbf{0}$ has a maximum at $\mathbf{x}^*$, then there exist $\mathbf{s}^* > \mathbf{0}$ and $\mathbf{y}^* \in \mathbb{R}^m$ such that $\mathbf{x}^*, \mathbf{y}^*, \mathbf{s}^*$ satisfy (7.4). Next, we formulate conditions for the existence of the maximum (we prove only their sufficiency, but it can be shown that they are also necessary), and we show that under these conditions, the maximum is characterized by (7.4).

7.2.1 Lemma. *Let us suppose that the linear program (7.2) has a feasible solution $\tilde{\mathbf{x}} > \mathbf{0}$ and that the dual linear program (7.5) has a feasible solution $\tilde{\mathbf{y}}$ such that the slack vector $\tilde{\mathbf{s}} = A^T \tilde{\mathbf{y}} - \mathbf{c}$ satisfies $\tilde{\mathbf{s}} > \mathbf{0}$. (Less formally, both the primal and dual linear programs have an interior feasible point.) Then for every $\mu > 0$ the system (7.4) has a unique solution $\mathbf{x}^* = \mathbf{x}^*(\mu)$, $\mathbf{y}^* = \mathbf{y}^*(\mu)$, $\mathbf{s}^* = \mathbf{s}^*(\mu)$, and $\mathbf{x}^*(\mu)$ is the unique maximizer of f_μ subject to $A\mathbf{x} = \mathbf{b}$ and $\mathbf{x} > \mathbf{0}$.*

Proof. Let $\mu > 0$ be fixed. We begin with the following claim.

Claim. Under the assumptions of the lemma, the set $Q = \{\mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x} > \mathbf{0}, f_\mu(\mathbf{x}) \geq f_\mu(\tilde{\mathbf{x}})\}$ is bounded.

Proof of the claim. We have

$$\begin{aligned}
 f_\mu(\mathbf{x}) &= \mathbf{c}^T \mathbf{x} + \mu \sum_{j=1}^n \ln x_j \\
 &= \mathbf{c}^T \mathbf{x} + \tilde{\mathbf{y}}^T (\mathbf{b} - A\mathbf{x}) + \mu \sum_{j=1}^n \ln x_j \quad (\text{since } A\mathbf{x} = \mathbf{b}) \\
 &= (\mathbf{c}^T - \tilde{\mathbf{y}}^T A) \mathbf{x} + \tilde{\mathbf{y}}^T \mathbf{b} + \mu \sum_{j=1}^n \ln x_j \\
 &= -\tilde{\mathbf{s}}^T \mathbf{x} + \tilde{\mathbf{y}}^T \mathbf{b} + \mu \sum_{j=1}^n \ln x_j \quad (\text{since } A^T \tilde{\mathbf{y}} - \tilde{\mathbf{s}} = \mathbf{c}) \\
 &= \tilde{\mathbf{y}}^T \mathbf{b} + \sum_{j=1}^n (\mu \ln x_j - \tilde{s}_j x_j).
 \end{aligned}$$

The first term of the last line is a constant, and the rest is a sum of univariate functions. Each of these univariate functions is of the form $h_\alpha(x) = \mu \ln x - \alpha x$ with $\mu, \alpha > 0$. Elementary calculus shows that $h_\alpha(x)$ attains a unique maximum at

$x = \frac{\mu}{\alpha}$, and in particular, it is bounded from above. Moreover, for every constant C , the set $\{x \in (0, \infty) : h_\alpha(x) \geq -C\}$ is bounded.

Setting $K = f_\mu(\tilde{\mathbf{x}}) - \tilde{\mathbf{y}}^T \mathbf{b}$, we have $Q \subseteq \{\mathbf{x} > \mathbf{0} : \sum_{j=1}^n h_{\tilde{s}_j}(x_j) \geq K\} \subseteq \prod_{j=1}^n \{x > 0 : h_{\tilde{s}_j}(x) \geq K - \sum_{i \neq j} \max_{z \in \mathbb{R}} h_{\tilde{s}_i}(z)\}$. The last set is a Cartesian product of bounded intervals and the claim is proved.

So the set Q is bounded, and it is not hard to check that it is closed. Hence the continuous function f_μ attains a maximum on it, which, as we know, is unique. This shows that f_μ attains a maximum under the assumptions of the lemma, and by means of Lagrange multipliers we have shown that this maximum yields a solution of (7.4). It remains to verify that this is the only solution of (7.4). What we do is to show that for every solution $\bar{\mathbf{x}}, \bar{\mathbf{y}}, \bar{\mathbf{s}}$ of (7.4), $\bar{\mathbf{x}}$ maximizes f_μ (we note that $\bar{\mathbf{s}}$ and $\bar{\mathbf{y}}$ are uniquely determined by $\bar{\mathbf{x}}$ through the relations $s_j x_j = \mu$ and $A^T \mathbf{y} - \mathbf{s} = \mathbf{c}$ from (7.4), using the assumption that A has full rank).

Let $\bar{\mathbf{x}}, \bar{\mathbf{y}}, \bar{\mathbf{s}}$ be a solution of (7.4) and let $\mathbf{x}$ satisfy $A\mathbf{x} = \mathbf{b}$ and $\mathbf{x} > \mathbf{0}$. Exactly as above we can express $f_\mu(\mathbf{x}) = \bar{\mathbf{y}}^T \mathbf{b} + \sum_{i=1}^m (\mu \ln x_j - \bar{s}_j x_j)$, and the right-hand side is maximized by setting $x_j = \mu/\bar{s}_j$, that is, for $\mathbf{x} = \bar{\mathbf{x}}$. The lemma is proved. $\square$

The set

$$\{(\mathbf{x}^*(\mu), \mathbf{y}^*(\mu), \mathbf{s}^*(\mu)) \in \mathbb{R}^{2n+m} : \mu > 0\}$$

is called the **primal–dual central path** of the linear program (7.2), and this is actually what the algorithm will follow (approximately).

The algorithm. The algorithm for solving the linear program (7.2) maintains current vectors $\mathbf{x}, \mathbf{s} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^m$, with $\mathbf{x} > \mathbf{0}$ and $\mathbf{s} > \mathbf{0}$, that satisfy all of the *linear* equations in (7.4); that is, $A\mathbf{x} = \mathbf{b}$ and $A^T \mathbf{y} - \mathbf{s} = \mathbf{c}$. This makes sense, since it is the quadratic equations $s_j x_j = \mu$ that make the problem complicated, and moreover, these are the only equations in (7.4) in which μ enters. (We are still postponing the question of obtaining the initial $\mathbf{x}, \mathbf{y}, \mathbf{s}$.)

The current $\mathbf{x}, \mathbf{y}, \mathbf{s}$ will in general fail to satisfy the conditions $s_j x_j = \mu$, since we follow the primal–dual central path only approximately. We need to quantify by *how much* they fail to satisfy them, and one suitable “centrality” measure turns out to be

$$\text{cdist}_\mu(\mathbf{x}, \mathbf{s}) = \left\| \left(\rho(s_1 x_1, \mu), \rho(s_2 x_2, \mu), \dots, \rho(s_n x_n, \mu) \right) \right\|,$$

where $\|\cdot\|$ is the Euclidean norm of a vector and $\rho(a, \mu) = \sqrt{a/\mu} - \sqrt{\mu/a}$. (This may look a little arbitrary, but the important thing is that it works.

Other variants of algorithms following the central path may use different distance notions.)

In a typical iteration of the algorithm, we have the current μ and $\mathbf{x}, \mathbf{y}, \mathbf{s}$ such that $\text{cdist}_\mu(\mathbf{x}, \mathbf{s})$ is sufficiently small; concretely, one can take the smallness condition to be $\text{cdist}_\mu(\mathbf{x}, \mathbf{s}) < \sqrt{2}$. Then we decrease μ slightly; in the considered algorithm we replace μ by $(1 - \frac{1}{2\sqrt{n}})\mu$. For this new μ we again want to approximate the solution $(\mathbf{x}^*(\mu), \mathbf{y}^*(\mu), \mathbf{s}^*(\mu))$ to (7.4) sufficiently closely. Since μ changed by only a little, we expect that the $\mathbf{x}, \mathbf{y}, \mathbf{s}$ from the previous iteration will be good initial guesses; in other words, in order to get $\mathbf{x}^*(\mu), \mathbf{y}^*(\mu), \mathbf{s}^*(\mu)$, we need to change $\mathbf{x}, \mathbf{y}, \mathbf{s}$ by only a little. Let us denote the required changes by $\Delta\mathbf{x}$, $\Delta\mathbf{y}$, and $\Delta\mathbf{s}$, respectively. So we look for $\Delta\mathbf{x}$, $\Delta\mathbf{y}$, $\Delta\mathbf{s}$ such that (7.4) is satisfied by $\mathbf{x} + \Delta\mathbf{x}$, $\mathbf{y} + \Delta\mathbf{y}$, and $\mathbf{s} + \Delta\mathbf{s}$; that is,

$$\begin{aligned} A(\mathbf{x} + \Delta\mathbf{x}) &= \mathbf{b} \\ A^T(\mathbf{y} + \Delta\mathbf{y}) - (\mathbf{s} + \Delta\mathbf{s}) &= \mathbf{c} \\ ((s_1 + \Delta s_1)(x_1 + \Delta x_1), \dots, (s_n + \Delta s_n)(x_n + \Delta x_n)) &= \mu \mathbf{1} \\ \mathbf{x} + \Delta\mathbf{x} > \mathbf{0}, \mathbf{s} + \Delta\mathbf{s} > \mathbf{0}. \end{aligned}$$

Using the fact that $\mathbf{x}, \mathbf{y}, \mathbf{s}$ satisfy the linear equations in (7.4) exactly, that is, $A\mathbf{x} = \mathbf{b}$ and $A^T\mathbf{y} - \mathbf{s} = \mathbf{c}$, the first two lines simplify to $A\Delta\mathbf{x} = \mathbf{0}$ and $A^T\Delta\mathbf{y} - \Delta\mathbf{s} = \mathbf{0}$. So far things were exact, but now we make a heuristic step: Since $\Delta\mathbf{x}$ and $\Delta\mathbf{s}$ are supposedly small compared to $\mathbf{s}$ and $\mathbf{x}$, we will neglect the second-order products $\Delta x_j \Delta s_j$ in the equation system from the third line. We will thus approximate the required changes in $\mathbf{x}$, $\mathbf{s}$, $\mathbf{y}$ by a solution to the following system:

$$\begin{aligned} A\Delta\mathbf{x} &= \mathbf{0} \\ A^T\Delta\mathbf{y} - \Delta\mathbf{s} &= \mathbf{0} \\ (s_1\Delta x_1 + x_1\Delta s_1, \dots, s_n\Delta x_n + x_n\Delta s_n) &= \mu\mathbf{1} - (s_1x_1, \dots, s_nx_n). \end{aligned} \tag{7.6}$$

The unknowns are $\Delta\mathbf{x}$, $\Delta\mathbf{y}$, $\Delta\mathbf{s}$, while $\mathbf{x}, \mathbf{y}, \mathbf{s}$ are regarded as constant. Hence this is a system of *linear* equations, and it is the system whose fast solution is a computational bottleneck of this interior point method. (For an actual computation the system can still be simplified by algebraic manipulations, but this is not our concern here.) In general we also need to worry about the positivity of $\mathbf{x} + \Delta\mathbf{x}$ and $\mathbf{s} + \Delta\mathbf{s}$, but in the algorithm considered below, luckily, it turns out that this is satisfied automatically—we will comment on this later.

It can be shown that passing from $\mathbf{x}, \mathbf{s}, \mathbf{y}$ to $\mathbf{x} + \Delta\mathbf{x}, \mathbf{y} + \Delta\mathbf{y}, \mathbf{s} + \Delta\mathbf{s}$, with $\Delta\mathbf{x}, \Delta\mathbf{y}, \Delta\mathbf{s}$ given by (7.6), can be regarded as a step of the Newton iterative method for solving the system (7.4).

We are ready to describe the algorithm. The input consists of a real $m \times n$ matrix A of rank m , vectors $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$, and a real $\varepsilon > 0$, which is a

parameter controlling the accuracy of the returned solution. (If we want an exact solution, the approximate solution found by the algorithm still has to be “rounded” suitably; we will not discuss this step.)

1. Set $\mu = 1$ and initialize $\mathbf{x}, \mathbf{y}, \mathbf{s}$ so that $A\mathbf{x} = \mathbf{b}$, $A^T\mathbf{y} - \mathbf{s} = \mathbf{c}$, $\mathbf{x} > \mathbf{0}$, $\mathbf{s} > \mathbf{0}$, and $\text{cdist}_\mu(\mathbf{x}, \mathbf{s}) < \sqrt{2}$.
2. (Main loop) While $\mu \geq \varepsilon$, repeat Steps 3 and 4. As soon as $\mu < \varepsilon$, return $\mathbf{x}$ as an approximately optimal solution and stop.
3. Replace μ with $\left(1 - \frac{1}{2\sqrt{n}}\right)\mu$.
4. (Newton step) Compute $\Delta\mathbf{x}$, $\Delta\mathbf{y}$, $\Delta\mathbf{s}$ as the (unique) solution of the linear system (7.6). Replace $\mathbf{x}$ by $\mathbf{x} + \Delta\mathbf{x}$, $\mathbf{y}$ by $\mathbf{y} + \Delta\mathbf{y}$, and $\mathbf{s}$ by $\mathbf{s} + \Delta\mathbf{s}$. Go to the next iteration of the main loop.

Step 1 is highly nontrivial—we know that finding a feasible solution of a general linear program is computationally as hard as finding an optimal solution—and we will discuss it soon. The rest of the algorithm has been specified completely, up to the way of solving (7.6) efficiently.

What needs to be done in the analysis. Here we prove neither correctness of the algorithm nor a bound on its running time. These things are not really difficult but they are somewhat technical. We just outline what has to be done.

Let us note that the centrality measure appears only in the first step of the algorithm (initialization). In the main loop we do not explicitly check that the current $\mathbf{x}$ and $\mathbf{s}$ stay close to the central path—this has to be established in the analysis, and a similar thing holds for the conditions $\mathbf{x} > \mathbf{0}$ and $\mathbf{s} > \mathbf{0}$. In other words, one needs to show that the following *invariant* holds for the current $\mathbf{x}$ and $\mathbf{s}$ in each iteration of the main loop:

Invariant: $\mathbf{x} > \mathbf{0}$, $\mathbf{s} > \mathbf{0}$, and $\text{cdist}_\mu(\mathbf{x}, \mathbf{s}) < \sqrt{2}$.

The next item is finiteness and convergence. It turns out that the algorithm always finishes, and moreover, it needs at most $O(\sqrt{n} \log \frac{1}{\varepsilon})$ iterations. Last but not least, there is also the issue of rounding errors and numerical stability. This concludes our sketch of the analysis.

Variations. The realm of interior point methods is vast, and even the number of papers on central path methods runs into the thousands. We mention just several ideas on how the described algorithm can be varied, with the aim of better practical convergence, numerical stability, etc.

- (Higher-order methods) We have said that the computation of $\Delta\mathbf{x}$, $\Delta\mathbf{y}$, $\Delta\mathbf{s}$ is equivalent to a step of the Newton method. This method locally approximates the considered functions by linear

functions, based on their first derivatives. One can also employ *higher-order methods*, where the approximation is done by multivariate polynomials, based on higher-order derivatives.

- (Truncated Newton steps) In the algorithm described above, luckily, it can be shown that making the full “Newton step,” i.e., going from $\mathbf{x}, \mathbf{y}, \mathbf{s}$ to $\mathbf{x} + \Delta\mathbf{x}, \mathbf{y} + \Delta\mathbf{y}, \mathbf{s} + \Delta\mathbf{s}$, cannot leave the feasible region. For other algorithms this need not be guaranteed, and then one chooses a parameter $\alpha \in (0, 1]$ in each iteration and moves only to $\mathbf{x} + \alpha\Delta\mathbf{x}, \mathbf{y} + \alpha\Delta\mathbf{y}, \mathbf{s} + \alpha\Delta\mathbf{s}$, where α is determined so as to maintain feasibility, or also depending on other considerations.
- (Long-step methods) Decreasing μ by the factor $1 - \frac{1}{2\sqrt{n}}$ is a careful, “short-step” strategy, designed so that we do not move too far along the central path and a small change again brings $\mathbf{x}, \mathbf{y}, \mathbf{s}$ close enough to the new point of the central path. In practice it seems advantageous to make *longer steps*, i.e., to decrease μ more significantly. For example, some algorithms go from μ to $\frac{1}{10}\mu$ or so. There are even adaptive algorithms (where the new μ is not given by an explicit formula) that asymptotically achieve quadratic convergence; that is, for μ sufficiently small, a single steps goes from μ to $\text{const} \cdot \mu^2$.

After such a large change of μ , it is in general not sufficient to make a single Newton step. Rather, one iterates Newton steps with μ fixed until the current $\mathbf{x}$ and $\mathbf{s}$ get sufficiently close to the central path.

The theoretical analysis becomes more difficult for long-step methods, and for some of the practically most successful such algorithms no reasonable theoretical bounds are known.

Initialization. It remains to say how the first step of the algorithm, finding the initial $\mathbf{x}, \mathbf{y}, \mathbf{s}$, can be realized. There are several approaches. Here we discuss one of them, an elegant method called a *self-dual embedding*.

The idea is this: Given an input linear program, we set up another, auxiliary linear program with the following properties:

- (P1) The auxiliary linear program is always feasible and bounded and there is a simple, explicitly specified vector lying on its central path, from which the above path-following algorithm can be started.
- (P2) From the optimal solution of the auxiliary linear program found by the algorithm we can read off an optimal solution of the original linear program or conclude that the original linear program is infeasible or unbounded.

We develop the auxiliary linear program in several steps. Here things come out more nicely if we start with the original linear pro-

gram in an inequality form:

$$\text{Maximize } \mathbf{c}^T \mathbf{x} \text{ subject to } A\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}. \quad (7.7)$$

As we have noted at the end of Section 6.1, the duality theorem implies that (7.7) is feasible and bounded if and only if the following system has a solution:

$$A\mathbf{x} \leq \mathbf{b}, \quad A^T \mathbf{y} \geq \mathbf{c}, \quad \mathbf{b}^T \mathbf{y} - \mathbf{c}^T \mathbf{x} \leq 0, \quad \mathbf{x}, \mathbf{y} \geq \mathbf{0}$$

(the first inequality is the feasibility of $\mathbf{x}$, the second inequality is the feasibility of $\mathbf{y}$ for the dual linear program, and the third inequality forces equality of the primal and dual objective functions). Now comes a small trick: We introduce a new scalar variable $\tau \geq 0$ (we will use Greek letters to denote such “stand-alone” scalar variables) and we multiply the right-hand sides by it. In this way, we obtain a *homogeneous* system (called the *Goldman–Tucker system*):

$$\begin{array}{rcl} A\mathbf{x} - \tau\mathbf{b} & \leq & \mathbf{0} \\ -A^T \mathbf{y} & + & \tau\mathbf{c} \leq \mathbf{0} \\ \mathbf{b}^T \mathbf{y} - \mathbf{c}^T \mathbf{x} & \leq & 0 \\ \mathbf{x}, \mathbf{y} & \geq & \mathbf{0}, \quad \tau \geq 0. \end{array} \quad (\text{GTS})$$

This system, being homogeneous, always admits the zero solution, but this is uninteresting. Interesting solutions, which allow us to solve the original linear program, are those with $\tau > 0$ or $\rho > 0$, where $\rho = \rho(\mathbf{x}, \mathbf{y}) = \mathbf{c}^T \mathbf{x} - \mathbf{b}^T \mathbf{y}$ denotes the slack in the last inequality. This is because of the following lemma:

7.2.2 Lemma. *No solution of (GTS) has both τ and ρ nonzero, and exactly one of the following possibilities always occurs:*

- (i) *There is a solution $(\mathbf{x}, \mathbf{y}, \tau)$ with $\tau > 0$ (and $\rho = 0$), in which case $\frac{1}{\tau}\mathbf{x}$ is an optimal solution of the primal linear program (7.7) and $\frac{1}{\tau}\mathbf{y}$ is an optimal solution of the dual.*
- (ii) *There is a solution $(\mathbf{x}, \mathbf{y}, \tau)$ with $\rho > 0$ (and $\tau = 0$), in which case the primal linear program (7.7) is infeasible or unbounded.*

Proof (sketch). By weak duality it is immediate that any solution with $\tau > 0$ has $\rho = 0$ and yields a pair of optimal solutions. Conversely, by the (strong) duality theorem, any pair $(\mathbf{x}^*, \mathbf{y}^*)$ of optimal solutions provides a solution of (GTS) with $\tau = 1$. Hence $\rho > 0$ implies $\tau = 0$ and infeasibility or unboundedness of the primal linear program. It remains to check that infeasibility of the primal linear program or of the dual linear program gives a solution of (GTS) with $\rho > 0$.

Let us assume, for example, that the primal linear program is infeasible (the dual case is analogous); that is, $A\mathbf{x} = \mathbf{b}$ has no nontrivial

nonnegative solution. Then by the Farkas lemma (Proposition 6.4.1) there is a $\bar{\mathbf{y}}$ with $A^T \bar{\mathbf{y}} \geq \mathbf{0}$ and $\mathbf{b}^T \bar{\mathbf{y}} < 0$, and then $\mathbf{x} = \mathbf{0}$, $\mathbf{y} = \bar{\mathbf{y}}$, $\tau = 0$ is a solution to (GTS) with $\rho = -\mathbf{b}^T \bar{\mathbf{y}} > 0$. $\square$

As a side remark, we note that applying the Farkas lemma to the Goldman–Tucker system yields an alternative derivation of the duality theorem from the Farkas lemma.

To boost the moral, we note that we have achieved something like (P2): By means of solutions to (GTS) of a special kind we can solve the original linear program, as well as deal with its infeasibility or unboundedness. But how do we compute a solution with $\rho > 0$ or $\tau > 0$, avoiding the trivial zero solution?

Luckily, interior point methods are very suitable for this, since they converge to a “most generic” optimal solution of the given linear program. Roughly speaking, the interior point algorithm described above converges to the analytic center of the set of all optimal solutions, and this particular optimal solution does not satisfy any constraint (inequality or nonnegativity constraint) with equality if this can be avoided at all. In particular, if we made (GTS) into a linear program by adding a suitable objective function, a “most generic” optimal solution would have $\tau > 0$ or $\rho > 0$.

There is still an obstacle to be overcome: We do not have an explicit feasible interior point for (GTS) to start the algorithm, and what is worse, no feasible interior point (one with all variables and slacks strictly positive) exists! This is because we always have $\tau = 0$ or $\rho = 0$, as was noted in the lemma.

The next step is to enlarge the system so that the vector of all 1’s is “forced” as a feasible interior point. Before proceeding we simplify the notation: We write the system (GTS) in matrix form as $M_0 \mathbf{u} \leq \mathbf{0}$, $\mathbf{u} \geq \mathbf{0}$, where

$$M_0 = \left(\begin{array}{c|c|c} 0 & A & -\mathbf{b} \\ \hline -A^T & 0 & \mathbf{c} \\ \hline \mathbf{b} & -\mathbf{c} & 0 \end{array} \right), \quad \mathbf{u} = \begin{pmatrix} \mathbf{y} \\ \mathbf{x} \\ \tau \end{pmatrix}.$$

We note that $M_0^T = -M_0$; that is, M_0 is a *skew-symmetric matrix*. If the original matrix A has size $m \times n$, then M_0 is a $k \times k$ matrix with $k = n + m + 1$.

Now we set up the appropriate linear program with a feasible interior point, and then we gradually explain how and why it works. We define a vector $\mathbf{r} \in \mathbb{R}^k$ by $\mathbf{r} = \mathbf{1} + M_0 \mathbf{1}$, a $(k+1) \times (k+1)$ matrix M by

$$M = \left(\begin{array}{c|c} M_0 & -\mathbf{r} \\ \hline \mathbf{r}^T & 0 \end{array} \right)$$

(we note that M is skew-symmetric too, which will be useful), and a vector $\mathbf{q} \in \mathbb{R}^{k+1}$ by $\mathbf{q} = (0, 0, \dots, 0, k+1)$. We consider the following linear program with variable vector $\mathbf{v} = (\mathbf{u}, \vartheta)$ (that is, $\mathbf{u}$ with a new variable ϑ appended to the end):

$$\text{Maximize } -\mathbf{q}^T \mathbf{v} \text{ subject to } M\mathbf{v} \leq \mathbf{q}, \mathbf{v} \geq \mathbf{0}. \quad (\text{SD})$$

First we explain the abbreviation (SD). It stands for *self-dual*, and indeed, if we form the dual linear program to (SD), we obtain

$$\text{minimize } \mathbf{q}^T \mathbf{w} \text{ subject to } M^T \mathbf{w} \geq -\mathbf{q}, \mathbf{w} \geq \mathbf{0}.$$

Using $M^T = -M$ and changing signs, which changes minimization to maximization and flips the direction of the inequality, we arrive exactly at (SD). So (SD) is equivalent to its own dual linear program.

For a feasible solution $\mathbf{v}$ of (SD) we define the *slacks* by $\mathbf{z} = \mathbf{z}(\mathbf{v}) = \mathbf{q} - M\mathbf{v}$ (“z for zacks”). Here is a key notion: A feasible solution $\mathbf{v}$ of (SD) is called **strictly complementary** if for every $j = 1, 2, \dots, k+1$ we have $v_j > 0$ or $z_j > 0$. The next lemma shows that a strictly complementary optimal solution is exactly what we need for solving the original linear program (7.7) (more precisely, we don’t need the full force of strict complementarity, only a strict complementarity for a particular j).

7.2.3 Lemma. *The linear program (SD) is feasible and bounded, every optimal solution has $\vartheta = 0$, and hence its $\mathbf{u}$ -part is a solution of the Goldman–Tucker system (GTS). Moreover, every strictly complementary optimal solution yields a solution of (GTS) with $\tau > 0$ or $\rho > 0$.*

Proof. We have $\mathbf{0}$ as a feasible solution of (SD), and also of its dual. Thus (SD) is feasible and bounded. For $\mathbf{v} = \mathbf{w} = \mathbf{0}$ both the primal and dual objective functions have value 0, so 0 must be their common optimal value. It follows that every optimal solution has $\vartheta = 0$. The rest of the lemma is easily checked. $\square$

Hence for solving the original linear program (7.7) it suffices to find a strictly complementary optimal solution of (SD). It turns out that this is exactly what the algorithm described above computes.

We will not prove this in full, but let us see what is going on. In order to apply the algorithm to (SD), we first need to convert (SD) to equational form by adding the slack variables:

$$\text{Maximize } -\mathbf{q}^T \mathbf{v} \text{ subject to } M\mathbf{v} + \mathbf{z} = \mathbf{q}, \mathbf{v}, \mathbf{z} \geq \mathbf{0}.$$

Now we want to write down the system (7.4) specifying points on the central path for the considered linear program. So we substitute

$A = (M \mid I_{k+1})$ (so A is a $(k+1) \times 2(k+1)$ matrix), $\mathbf{b} = \mathbf{q}$, $\mathbf{c} = (-\mathbf{q}, \mathbf{0})$, and $\mathbf{x} = (\mathbf{v}, \mathbf{z})$. We also need to introduce the extra variables $\mathbf{y}$ and $\mathbf{s}$ appearing in (7.4): For reasons that will become apparent later, we write $\mathbf{v}'$ for $\mathbf{y}$ and $(\mathbf{z}', \mathbf{z}'')$ for $\mathbf{s}$, where $\mathbf{z}'$ and $\mathbf{z}''$ are $(k+1)$ -component vectors. So we have $\mathbf{v}, \mathbf{z}, \mathbf{z}', \mathbf{z}''$ nonnegative, while $\mathbf{v}'$ is (so far) arbitrary.

The equation $A\mathbf{x} = \mathbf{b}$ from (7.4) becomes $M\mathbf{v} + \mathbf{z} = \mathbf{q}$. From $A^T\mathbf{y} - \mathbf{s} = \mathbf{c}$ we get two equations: $M^T\mathbf{v}' - \mathbf{z}' = -\mathbf{q}$ and $\mathbf{v}' - \mathbf{z}'' = \mathbf{0}$. The first of these two can be rewritten as $M\mathbf{v}' + \mathbf{z}' = \mathbf{q}$. The second just means that $\mathbf{v}' = \mathbf{z}'' \geq \mathbf{0}$, and so $\mathbf{z}''$ can be disregarded if we add the constraint $\mathbf{v}' \geq \mathbf{0}$. Finally, $s_j x_j = \mu$ in (7.4) yields $v_j z'_j = \mu$ and $v'_j z_j = \mu$ for all $j = 1, 2, \dots, k+1$. The full system is thus

$$\begin{aligned} M\mathbf{v} + \mathbf{z} &= \mathbf{q} \\ M\mathbf{v}' + \mathbf{z}' &= \mathbf{q} \\ v_j z'_j &= \mu \text{ for all } j = 1, 2, \dots, k+1 \\ v'_j z_j &= \mu \text{ for all } j = 1, 2, \dots, k+1 \\ \mathbf{v}, \mathbf{z}, \mathbf{v}', \mathbf{z}' &\geq \mathbf{0}. \end{aligned}$$

It is easily verified, using the skew-symmetry of M_0 , that the system $M\mathbf{v} + \mathbf{z} = \mathbf{q}$ is satisfied by $\mathbf{v} = \mathbf{z} = \mathbf{1}$ (M and $\mathbf{q}$ were set up that way). Thus $\mathbf{v} = \mathbf{z} = \mathbf{v}' = \mathbf{z}' = \mathbf{1}$ is a solution of the just-derived system with $\mu = 1$, and we can use it as an initial point on the central path in Step 1 of the algorithm. To complete the analysis one needs to show that the algorithm converges to a strictly complementary optimal solution, and as we said above, this part is omitted.

As a final remark to the algorithm we note that the system above can be simplified. We know that (7.4) in general has a unique solution (provided that the primal and dual linear programs both have a feasible interior point, which is satisfied in our case). At the same time, we observe that if $\mathbf{v}, \mathbf{z}, \mathbf{v}', \mathbf{z}'$ is a solution, then interchanging $\mathbf{v}$ with $\mathbf{v}'$ and $\mathbf{z}$ with $\mathbf{z}'$ also yields a solution, and so uniqueness implies $\mathbf{v} = \mathbf{v}'$ and $\mathbf{z} = \mathbf{z}'$. Therefore, it is sufficient to work with the simpler system

$$M\mathbf{v} + \mathbf{z} = \mathbf{q}, \quad v_j z_j = \mu \text{ for } j = 1, 2, \dots, k+1, \quad \mathbf{z}, \mathbf{v} \geq \mathbf{0}$$

and to set up the corresponding linear system for the changes $\Delta\mathbf{v}$ and $\Delta\mathbf{z}$ accordingly. This concludes the description and partial analysis of the considered interior point algorithm.

Let us remark that the self-dual embedding trick is quite universal: Almost any interior point algorithm can be used for computing an optimum of the self-dual linear program constructed as above, and this yields an optimal solution of the original linear program.

Computational complexity of interior point methods. Several interior point methods for linear programming are known to be (weakly) polynomial

algorithms, including the one given above (with the self-dual embedding). The total number of bit operations for the best of these algorithms is bounded by $O(n^3 L)$, where L is the maximum bit size of coefficients in the linear program and n is the number of variables. The maximum number of iterations before reaching an optimum is $O(\sqrt{n}L)$. On the other hand, examples are known (based on the Klee–Minty cube, again!) for which any algorithm following the central path must make $\Omega(\sqrt{n} \log n)$ iterations; see

A. Deza, E. Nematollahi, and T. Terlaky: How good are interior point methods? Klee–Minty cubes tighten iteration-complexity bounds, Technical Report, McMaster University, 2004.

In practice the number of iterations seems to be bounded by a constant or by $O(\log n)$ in almost all cases. This is somewhat similar to the situation for the simplex method, where the worst-case behavior is much worse than the behavior for typical inputs.

Our presentation of interior point methods is inspired mostly by

T. Terlaky: An easy way to teach interior-point methods, *European Journal of Operational Research* 130, 1(2001), 1–19,

and full proofs of the results in this section can be found in

C. Roos, T. Terlaky, and J.-P. Vial: *Interior Point Methods for Linear Optimization (2nd edition)*, Springer, Berlin etc., 2005.

A compact survey is

F. A. Potra, S. J. Wright: Interior-point methods, *Journal of Computational and Applied Mathematics* 124(2000), pages 281–302;

at time of writing this text it was accessible at websites of the authors. Several more books from the late 1990s are cited in these papers, and an immense amount of material can be found online.